

UART Verification Plan

Parameterized 8-bit UART (TX/RX loopback) · SystemVerilog / UVM (IEEE 1800.2)

Field	Value
Design	Parameterized 8-bit UART, TX/RX internal loopback
Environment	SystemVerilog / UVM, IEEE 1800.2
Simulators	Verilator 5.050 and Questa
Configurations	4 baud divisors (9600 / 19200 / 57600 / 115200 @ 100 MHz)
Checkers	Queue-based scoreboard + 7 bound assertions
Coverage	5 cover properties, 5/5 closed at every divisor
Status	Closed — 0 errors, 0 fatals on both simulators

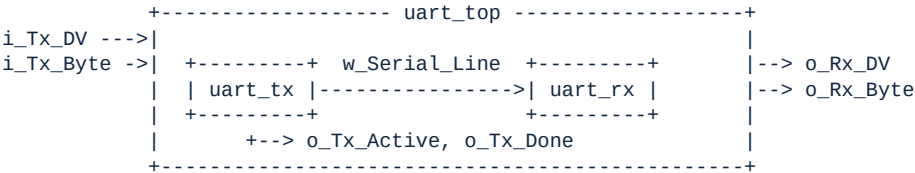
1. Scope

This document is the verification plan for the UART loopback DUT in rtl/. It defines what is verified, how each feature is stimulated and checked, the assertion and coverage goals, and what has deliberately been left out of scope.

The transmitter and receiver are third-party RTL (see THIRD_PARTY_NOTICES.md). The loopback wrapper, verification environment, assertion checker and build flow are original work.

2. Design under test

2.1 Topology



The transmitter's serial output is wired directly to the receiver's serial input inside the wrapper. There is no serial pin at the DUT boundary, which has two consequences: stimulus is applied as parallel bytes with the serial waveform observable only internally via w_Serial_Line; and the receiver cannot be stimulated independently, so line-level error injection is out of scope (see §9).

2.2 Parameters and baud divisors

Baud rate	CLKS_PER_BIT	Note
9600	10417	Slowest configuration, longest simulation
19200	5208	
57600	1736	
115200	868	Default for single-run targets

2.3 Interface

Signal	Dir	W	Description
i_Clock	in	1	System clock, 100 MHz in this environment
i_Tx_DV	in	1	Transmit data valid, sampled in s_IDLE
i_Tx_Byte	in	8	Payload, latched on s_IDLE → s_TX_START_BIT
o_Tx_Active	out	1	High for the duration of a transmitted frame
o_Tx_Done	out	1	Frame complete (see note in §6, A7)
o_Rx_DV	out	1	Receive data valid, one clock wide
o_Rx_Byte	out	8	Received payload

There is no reset in this RTL. The interface declares an unused `reset` placeholder; the FSMs initialise from their declared register values.

2.4 Transmitter FSM

State	Encoding	Behaviour
s_IDLE	3'b000	Line high; samples i_Tx_DV; latches payload
s_TX_START_BIT	3'b001	Line low for CLKS_PER_BIT clocks
s_TX_DATA_BITS	3'b010	8 bits LSB-first, CLKS_PER_BIT clocks each
s_TX_STOP_BIT	3'b011	Line high for CLKS_PER_BIT clocks
s_CLEANUP	3'b100	Holds r_Tx_Done, returns to s_IDLE

Frame format is 8-N-1: one start bit, eight data bits, one stop bit, no parity.

2.5 Receiver

The receiver double-registers the serial input to cross clock domains, detects the falling edge of the start bit, then waits $(\text{CLKS_PER_BIT} - 1) / 2$ clocks to align sampling to the centre of each bit period. This mid-bit alignment means the receiver completes its frame roughly half a bit period after the transmitter completes its own.

3. Verification environment

3.1 Component list

Component	File	Role
uart_item	tb/uvm/uart_item.sv	Transaction: payload byte + inter-frame delay
uart_stress_sequence	tb/uvm/uart_sequence.sv	Directed corner cases + weighted random stress
uart_driver	tb/uvm/uart_driver.sv	Drives tx_dv / tx_byte, waits on tx_done, applies delay
uart_monitor	tb/uvm/uart_monitor.sv	Two forked threads: TX sampling and RX sampling
uart_scoreboard	tb/uvm/uart_scoreboard.sv	Queue-based compare, dual analysis imports
uart_agent	tb/uvm/uart_agent.sv	Active/passive; sequencer + driver + monitor
uart_env	tb/uvm/uart_env.sv	Agent + scoreboard, TLM connections
uart_test	tb/uvm/uart_test.sv	Objection handling, sequence start
uart_sva_checker	tb/sva/uart_sva_checker.sv	7 assertions + 5 cover properties, bound to uart_top

3.2 Transaction fields

Field	Type	Rand	Description
data	bit [7:0]	yes	Payload byte
delay	int	yes	Idle clocks inserted after the frame drains
pick	int unsigned	yes	Uniform [0:99] selector shaping the delay distribution

pick exists for solver portability — see §8.

3.3 Stimulus profile

60 transactions per run, in three phases:

Phase	N	Payload	Delay	Intent
1	5	8'h00	0	Start bit adjacent to a 0 data bit; back-to-back framing
2	5	8'hFF	[1:20]	Last data bit adjacent to the stop bit
3	50	random	70% 0 / 20% [1:20] / 10% 1000	Mixed burst, gap and long-hold traffic

All three phases go through `randomize()`; the directed phases pin values with inline `randomize()` with rather than post-create field assignment, so the class constraints stay live rather than becoming dead code.

4. Checking strategy

Two independent mechanisms, deliberately checking different things.

Scoreboard — data integrity. The monitor's TX thread pushes every accepted byte onto a queue; the RX thread pops and compares. Because one scoreboard needs two write methods, `uvm_analysis_imp_decl(_sent)` and `(_got)` create two distinct analysis imports. A receive with an empty queue is a `UVM_ERROR`, as is a payload mismatch. The queue is order-dependent, which is sound here because the DUT holds a single frame in flight: the transmitter cannot begin a new frame until the previous one drains, so bytes cannot be reordered.

Assertions — bit-level timing. The scoreboard proves the right byte came out. It cannot prove the byte was framed correctly on the wire: a transmitter with a half-length start bit paired with a receiver sampling at the same wrong offset would still pass a data comparison. The bound checker watches `w_Serial_Line` and the TX FSM

directly and proves the framing that produced the data.

5. Testplan

ID	Feature	Stimulus	Check	Status
T1	Basic TX→RX byte transfer	Any phase	Scoreboard compare	Pass
T2	All-zeros payload	Phase 1	Scoreboard + A1–A3, C1	Pass
T3	All-ones payload	Phase 2	Scoreboard + A4, C2	Pass
T4	Back-to-back frames	Phase 1, Phase 3 (70%)	Scoreboard + A6, C3	Pass
T5	Long idle between frames	Phase 3 (10%)	Scoreboard + A5, C4	Pass
T6	Start-bit framing	All	A1, A2	Pass
T7	Per-bit timing vs CLKS_PER_BIT	All	A3	Pass
T8	Stop-bit framing	All	A4	Pass
T9	Idle line state	All	A5	Pass
T10	Payload integrity mid-frame	Phase 1, Phase 3	A6	Pass
T11	o_Tx_Done handshake width	All	A7	Pass
T12	Baud-rate parameterisation	Regression sweep	All of the above, ×4	Pass

6. Assertion plan

All assertions live in `uart_t_sva_checker`, bound into `uart_t_top`, with hierarchical taps into TX_INST for FSM state, bit counter and latched payload.

ID	Name	Property
A1	a_start_bit_low	$\$rose(tx_active) \Rightarrow (tx_serial == 0)$
A2	a_start_bit_duration	FSM cannot leave s_TX_START_BIT before the counter reaches CLKS_PER_BIT-1
A3	a_tx_bit_stable	In s_TX_DATA_BITS with a non-zero bit counter, the line is stable next clock
A4	a_stop_bit_high	In s_TX_STOP_BIT past the first clock, the line is high
A5	a_line_idle_high	In s_IDLE, the line is high next clock
A6	a_no_frame_overlap	While tx_active, the latched payload is stable
A7	a_tx_done_pulse	On $\$fell(tx_done)$, it was high for exactly the previous two clocks

Note on A7. The vendor header comment states that `o_Tx_Done` is driven high for one clock cycle. It is not. `r_Tx_Done` is set on the final `s_TX_STOP_BIT` clock *and* again in `s_CLEANUP`, and is cleared only on return to `s_IDLE`, giving a two-clock pulse. A7 pins the observed behaviour rather than the documented one. The discrepancy was found by writing the assertion from the specification and watching it fail.

7. Coverage plan

ID	Cover property	Goal	Observed (BAUD=868)
C1	c_payload_all_zeros	≥ 1	5
C2	c_payload_all_ones	≥ 1	5
C3	c_burst_back_to_back	≥ 1	42
C4	c_long_idle_gap	≥ 1	6

ID	Cover property	Goal	Observed (BAUD=868)
C5	c_rx_frame_complete	≥ 1	60

Closure: 5/5 at all four divisors.

C3 and C4 exist specifically to prove the weighted delay distribution reached both extremes rather than clustering in the middle. Without them, a solver that quietly collapsed the distribution to a single value would still produce a passing regression. Hit counts are accumulated in a plain always block mirroring the cover conditions and printed at end of simulation, so the numbers appear in the log on both simulators.

8. Tool portability constraints

Four limitations shaped how this environment is written. Each was worked around without weakening what is checked.

Constraint	Impact	Resolution
## N unsupported in sequence expressions	A2 and A7 could not be written with cycle delays	A2 rewritten as an FSM-state guard; A7 rewritten with \$past(tx_done, 1..3)
Cover-property action blocks crash the compiler (V3Localize)	Hit counters could not be attached to covers	Covers left bare for the vendor tool's native database; counters mirrored in an always block
UVM DPI layer has no open-source backend	Build fails with #error "hdl vendor backend is missing" before the testbench compiles	Build with +define+UVM_NO_DPI; two benign warnings result
dist cannot be composed with an inline equality	randomize() with { delay == 0; } reported unsatisfiable	Weighting derived from uniform pick plus implication constraints

The assertion source is restricted to |->, |==>, \$rose, \$fell, \$stable and \$past — the subset both simulators accept. Avoided: [*N] repetition, unbounded ranges, sequence local variables, and throughout / within / intersect.

A further trap sits in the stimulus rather than the assertions: non-blocking assignments reached from an initial block are demoted to blocking assignments by the open-source simulator. Driving the DUT exactly on the clock edge therefore races the RTL's own sampling and hangs the simulation on the first @(posedge tx_done). The assertion harness drives at #1 past the edge to avoid this.

9. Out of scope and future work

Recorded deliberately, so the coverage claim is not read as broader than it is.

Item	Reason / next step
Line-level error injection	The loopback wrapper offers no serial pin, so framing errors, baud mismatch, break conditions and glitches cannot be injected. Closing this needs either w_Serial_Line exposed at the wrapper boundary or a separate receiver-only environment with a serial driver.
Parity	The DUT implements 8-N-1 only; there is no parity logic to verify.
Reset	The RTL has no reset port, so reset-during-frame behaviour is untestable as designed.
Functional covergroups	Coverage is cover-property based. A covergroup on payload ranges, delay bins and their cross would give a stronger closure argument than five cover properties.
Register abstraction	No register interface on this DUT, so no RAL model applies.
Single test	One test class drives all stimulus. Splitting into directed and random tests over a common base would scale better.
Monitor sampling	The monitor samples on @(posedge clk) without a clocking block. Deterministic for this design because all DUT outputs are non-blocking-assigned, but a clocking block would make the sampling contract explicit.

10. Results summary

Flow	Configurations	Result
UVM regression, Verilator 5.050	4 divisors	0 UVM_ERROR, 0 UVM_FATAL

Flow	Configurations	Result
UVM regression, Questa	4 divisors	0 UVM_ERROR, 0 UVM_FATAL
Assertion harness, Verilator	4 divisors	RESULT: PASS, 5/5 covers

Per run, on both simulators: 60 driver transactions, 120 monitor observations, 120 scoreboard reports, no assertion failures. Identical transaction counts across two independent tools is the strongest single piece of evidence in this environment — it confirms the behaviour is a property of the design and testbench, not of one simulator's scheduler.